

Filler-ASR: a Self-Attention Read-out Head on a Frozen HuBERT-xlarge

A detailed, intuitive walkthrough of `filler_sa_reference.py` and the full-LibriSpeech training
`run_full960_sa8_pe_fps25_lin_100ep_bs32acc2` | July 2026

	TL;DR
Idea	Instead of CTC, treat ASR as per-frame classification: pack the transcript characters at the head of the frame axis, label every remaining frame with a special <code><fill></code> class, and train a small self-attention head with plain cross-entropy on top of a completely frozen HuBERT-xlarge encoder.
Trainable	8 post-norm SA layers + linear head = 157.46M params (~14%); the ~964M HuBERT body is frozen
Data	LibriSpeech 960h - 281,241 train utterances; dev-clean (500 clips) for model selection
Compute	6-GPU DDP, bf16 autocast, effective batch 384, 100 epochs = 73,200 optimizer steps
Result	test-clean (2,620 utts): WER 12.54% / CER 5.66% / SER 36.0% with greedy, <code></s></code> -capped decode (best dev WER 13.5% at step 40k)

1 The big picture - what problem is this solving, and how

Modern ASR on top of a self-supervised encoder (wav2vec2 / HuBERT) almost always uses **CTC**: the model emits a distribution per frame, and a dynamic-programming loss marginalises over all monotonic alignments between frames and characters. This project asks a deliberately different question: *can we skip alignment entirely and make ASR a plain per-frame classification problem?*

The trick is the `<fill>` **token**. The 26 letters, the word separator `|`, and a few specials form a 33-class vocabulary. For each clip we build a target sequence exactly as long as the encoder output (one label per 20 ms frame): the transcript characters are packed at the **head** of the frame axis - character *i* is pinned to frame *i* - then a `</s>` marker, then `<fill>` everywhere else. Every frame now has exactly one correct answer, so the loss is ordinary cross-entropy. No CTC blank, no forward-backward, no alignment lattice.

Why bother? Three reasons:

- **Probing:** with HuBERT completely frozen, whatever WER this reaches measures how linearly-accessible the transcript is from the encoder's last layer - the head cannot re-learn acoustics.
- **Cheapness:** only the head trains (~14% of parameters). The frozen encoder needs no backward pass and no stored activations, so memory is roughly forward-only.
- **Simplicity:** the whole learning signal is a softmax + CE per frame - trivially debuggable, and the single-file reference reproduces it end-to-end.

The catch is that this target is **positional, not acoustic**. The word spoken in second 8 of a clip contributes its letters near frame ~120, not near frame ~400 where its sound lives. A per-frame linear classifier on HuBERT features therefore *cannot* solve it (an earlier head-only experiment in this project sat at WER close to 1.0). Making it work needed three additions, and each one maps to a block in the reference file:

- **Sinusoidal positional encoding** (Block 1) - the head must know 'which slot am I?' to place characters by index.
- A **duration budget** in the loss (Block 4) - only the first `ceil(dur x 25)` frames are graded, so the model is not forced to emit `<fill>` over the entire long tail.
- A stack of **8 trainable self-attention layers** (Block 2) - enough capacity to *transport* acoustic evidence from where sounds occur to the head-of-sequence slots where labels sit.

2 End-to-end pipeline and the flow of dimensions

The whole system is one diagram. Blue is frozen, orange trains, tan is data preparation, purple is the loss path. Shapes use B = batch, L = audio samples, T = encoder frames.

filler_asr SA pipeline — frozen HuBERT tap → PE → SA×8 → lm_head, framewise CE

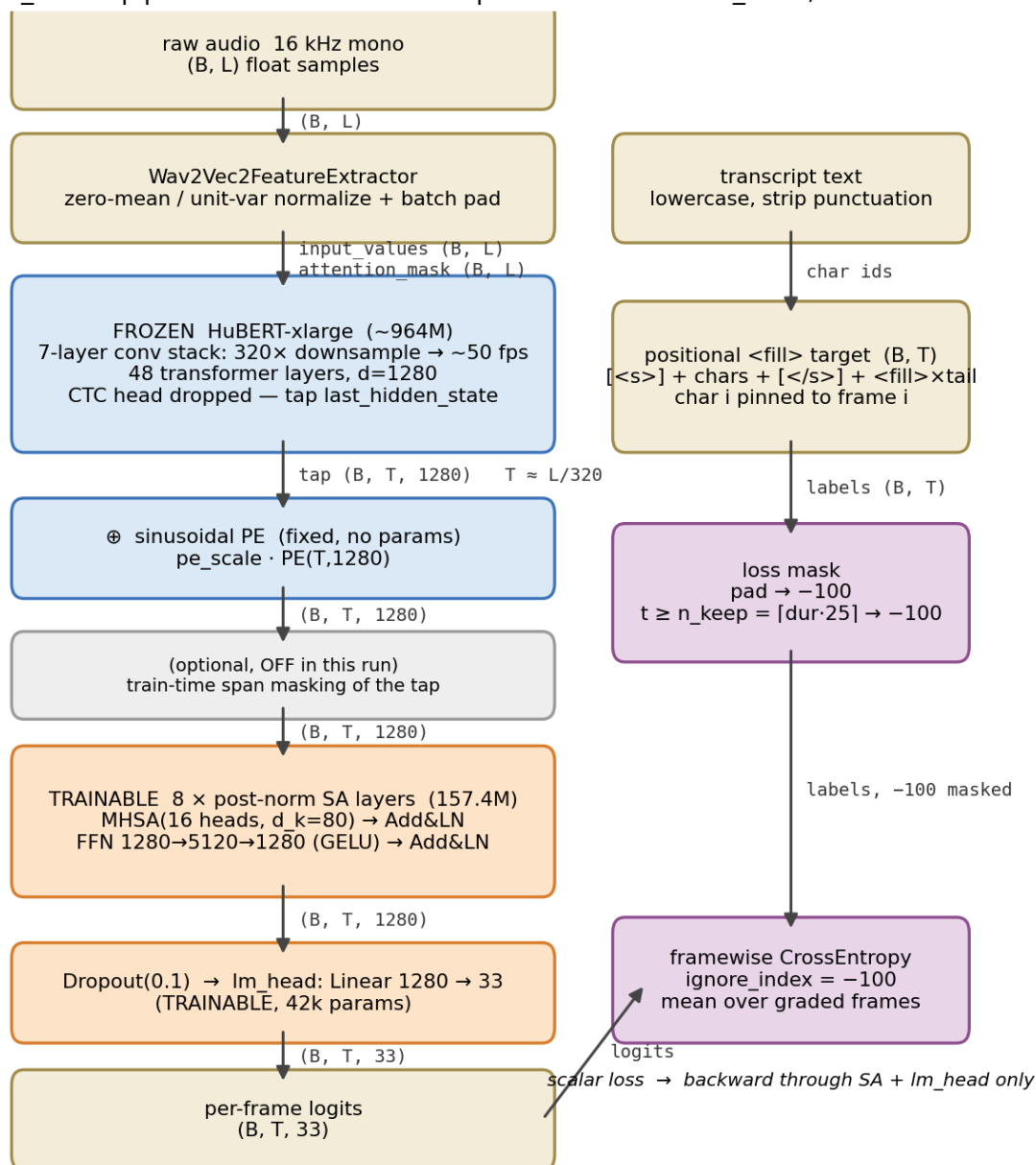


Figure 1 - the full pipeline of filler_sa_reference.py. Left column: the model forward path with tensor shapes. Right column: label preparation in the collator and the framewise CE loss. Only the orange blocks receive gradients.

A real worked example. Clip 1089-134686-0000 from test-clean is 10.44 s of 16 kHz audio. Following it through every stage:

stage	tensor / value	how it is computed
raw audio	L = 167,040 samples	10.44 s x 16,000 Hz
feature extractor	(1, 167040) + attention_mask	zero-mean/unit-variance normalise, batch pad

stage	tensor / value	how it is computed
conv stack (7 layers)	167040 -> 33407 -> 16703 -> 8351 -> 4175 -> 2087 -> 1043 -> T = 521	floor((L-k)/s)+1 per layer; 320x total downsample
HuBERT tap	(1, 521, 1280)	last_hidden_state of the frozen 48-layer encoder
+ sinusoidal PE	(1, 521, 1280)	adds PE(521, 1280), fixed, no parameters
8 x SA layers	(1, 521, 1280)	shape preserved; content mixed across time
lm_head	logits (1, 521, 33)	Linear 1280 -> 33 per frame
labels	(1, 521) ints	<s> + 155 chars + </s> = 157 head labels, then <fill>
duration budget	n_keep = ceil(10.44 x 25) = 261	labels[261:] = -100 (ignored in loss)
graded frames	261 of 521	157 content + 104 graded <fill>

Note the asymmetry that defines this whole design: the encoder produces ~50 frames per second, but read speech carries only ~12-18 characters per second. So even with one character per frame, the transcript occupies well under half of the frame axis - the rest is <fill> by construction. Everything else in the file (budget, masks, capped decoding) manages that tail.

3 Block 0 - vocabulary and frame geometry

The 33-token vocabulary. A character tokenizer (Wav2Vec2CTCTokenizer) over LibriSpeech text, plus <fill> appended as an extra special token:

id	token(s)	meaning
0	'	apostrophe (it's, don't)
1-26	a ... z	letters
27		word delimiter - spaces become before tokenising
28 / 29	<unk> / <pad>	unknown / padding
30 / 31	<s> / </s>	transcript start / end markers - </s> doubles as the model's own 'speech ended here' estimate at decode time
32	<fill>	the filler class: 'no character is assigned to this frame'

Frame geometry. compute_output_length() replays the HuBERT/Wav2Vec2 convolutional feature extractor arithmetic to predict exactly how many frames T the encoder will emit for L input samples - the label vector must be sized to T *before* the model runs:

$$L_{\text{out}} = \left\lfloor \frac{L_{\text{in}} - k}{s} \right\rfloor + 1 \quad (k, s) \in \{(10, 5), (3, 2), (3, 2), (3, 2), (3, 2), (2, 2), (2, 2)\} \quad T \approx \frac{L}{320} \quad (\approx 50 \text{ frames/s})$$

Seven conv layers with kernels [10,3,3,3,3,2,2] and strides [5,2,2,2,2,2,2] multiply to a 320x downsample: 16,000 samples/s / 320 = 50 frames/s, i.e. one frame per 20 ms. This formula is why the worked example above lands on exactly T = 521 - and the collator computes the same number independently so labels and logits line up frame-for-frame.

4 Block 1 - sinusoidal positional encoding (the +PE in the diagram)

Self-attention is **permutation-equivariant**: shuffle the input frames and the outputs shuffle identically. Nothing in Q.K dot products knows *where* a frame sits. For most ASR losses that is survivable (CTC only needs order via the encoder's own conv position information), but here it is fatal: the target says 'character 12 lives at frame 12', so the head must be able to address frames *by absolute index*. The fix is the classic Vaswani-2017 fixed encoding, added once to the tapped features:

$$PE_{p, 2i} = \sin\left(\frac{p}{10000^{2i/d}}\right) \quad PE_{p, 2i+1} = \cos\left(\frac{p}{10000^{2i/d}}\right) \quad \lambda_i = 2\pi \cdot 10000^{2i/d} \in [2\pi, 2\pi \cdot 10^4]$$

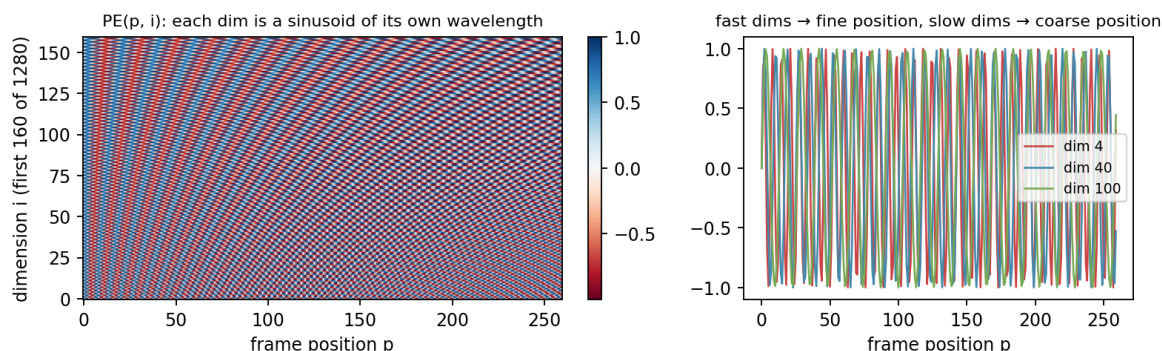


Figure 2 - left: the PE matrix; every dimension i oscillates with its own wavelength. Right: three sample dimensions - fast ones distinguish neighbouring frames, slow ones encode coarse position.

Intuition - a multi-hand clock. Each pair of dimensions (sin, cos) is a hand rotating at its own speed: dimension 0 completes a cycle every ~6 frames (a 'second hand'), the last dimensions take tens of thousands of frames (an 'hour hand'). Reading all 1280 hands together gives every frame position a unique, smoothly-varying fingerprint. Two useful properties:

- **Bounded and comparable:** values live in $[-1, 1]$, the same scale as layer-normalised features, so a plain addition works (scaled by `pe_scale`, 1.0 here).
- **Relative offsets are easy:** $PE(p+k)$ is a fixed linear function (a rotation) of $PE(p)$, so attention can learn 'look k frames to the right' as easily as 'look at frame 12'.

Because it has **zero parameters** it costs nothing and generalises to any length. In the code it is gated by `config.use_sinusoidal_pe` so older no-PE checkpoints still load; this run trains with PE on `(- -pe)`. One subtle ordering detail: the optional input masking (Section 7) happens *before* the PE add, so a masked frame keeps its position fingerprint while losing its content - exactly what a fill-in-from-context objective wants.

5 Block 2 - the model: frozen tap, 8 SA layers, lm_head

(a) The frozen tap. `FillerHubertSAModel` embeds a full `HubertModel` (conv extractor + 48 transformer layers, $d = 1280$) and simply reads its `last_hidden_state` - shape $(B, T, 1280)$. The original CTC head of the `hubert-xlarge-ls960-ft` checkpoint is never instantiated. Every HuBERT parameter has `requires_grad=False`, which has a second, quieter benefit: autograd stores no activations inside the encoder, so the 1B-parameter body costs only forward memory. The encoder is a pure feature pump.

(b) The trainable self-attention stack. Eight standard `nn.TransformerEncoderLayers` in **post-norm** arrangement (matching HuBERT's own encoder style), 16 heads, FFN width 5120, dropout 0.1, GELU. Per layer, per frame:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad X \in \mathbb{R}^{T \times 1280}, \quad W_{Q,K,V} \in \mathbb{R}^{1280 \times 1280} \quad (16 \text{ heads}, d_k = 80)$$

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V \quad M_{tj} = -\infty \text{ if } j \text{ is a pad frame, else } 0$$

Each of the 16 heads projects the 1280-dim frames into its own 80-dim subspace, scores every frame against every other frame (the $T \times T$ matrix $QK^\top / \sqrt{80}$), softmaxes those scores into attention weights, and averages the value vectors with them. The additive mask M implements the **frame mask**: batch-padding frames get a score of -infinity, i.e. zero attention weight, so short clips in a batch are unaffected by their padding. The 16 head outputs are concatenated back to 1280 dims and mixed by an output projection. Then the residual + LayerNorm + FFN sandwich:

$$x' = \text{LN}(x + \text{MHSA}(x)) \quad y = \text{LN}(x' + \text{FFN}(x')) \quad \text{FFN}(u) = \text{GELU}(uW_1 + b_1)W_2 + b_2$$

Division of labour intuition: MHSA is the *transport* layer - it is the only place information crosses time, and with 8 stacked layers evidence can hop from an acoustic frame to its distant positional slot in several attention steps. The FFN is per-frame *computation* - 3.9x wider than the model dim, it does the local 'given what was gathered here, which character is this?' classification work.

(c) Head and parameter arithmetic. After the stack: `Dropout(0.1)` then `lm_head = Linear(1280, 33)` per frame. Counting parameters ($d = 1280$, $f = 5120$):

$$P_{\text{layer}} = (4d^2 + 5d)_{\text{MHSA}} + (2df + f + d)_{\text{FFN}} + (4d)_{2\text{LN}} = 19,677,440 \quad (d = 1280, f = 5120)$$

$$8 \times 19,677,440 + (1280 \times 33 + 33)_{\text{lm head}} = 157,461,793 \approx 157.46\text{M trainable} \quad (\text{vs } \sim 964\text{M frozen HuBERT})$$

This matches the training log line `trainable=157.46M` exactly. It also explains the checkpoint size: $157.46\text{M floats} \times 4 \text{ bytes} = 630 \text{ MB}$ of weights, plus two AdamW moment buffers of the same size = $\sim 1.9 \text{ GB}$, which is precisely what `best.pt` weighs on disk. (The docstring's ' $\sim 39.4\text{M}$, $\sim 4\%$ ' figure is for the 2-layer reference default; this run trains 8 layers.)

(d) A real-world initialisation bug-fix worth knowing. HuggingFace's `_init_weights` for HuBERT does not know about `nn.MultiheadAttention`'s fused `in_proj_weight` parameter, so when `from_pretrained` re-initialises the keys missing from the checkpoint (all the new SA layers), those tensors would be left as allocated garbage. The class overrides `_init_weights` to Xavier-init them (and the optional `mask_embed`) explicitly - without this the head trains from NaN-ish weights.

(e) Freezing helpers. `unfreeze_all_except_feature_extractor()` freezes everything, then re-enables the SA stack + `lm_head`. An optional `unfreeze_top_hubert=N` argument additionally opens the top N HuBERT layers for gradual unfreezing (with a discriminative, smaller LR in `train.py`) - unused in this run: the whole body stayed frozen.

6 Blocks 3-4 - label preparation: the heart of the method

The collator `DataCollatorFillerASR` turns raw (audio path, text) rows into model-ready batches. Per clip it does four things: load + normalise audio, build the positional target, compute the duration budget, and pad everything into batch tensors.

Text normalisation. Strip punctuation `[ , ? . ! - ; : " ]`, lowercase, replace spaces with the word delimiter `|`. LibriSpeech references are already clean, so this mainly guarantees train/eval symmetry.

The positional target. With T frames (from the Block-0 formula) and N normalised characters:

$$y_0 = \langle s \rangle \quad y_t = c_t \quad (1 \leq t \leq N) \quad y_{N+1} = \langle /s \rangle \quad y_t = \langle \text{fill} \rangle \quad (N+1 < t < T) \quad N = \# \text{chars} \text{ (spaces} \rightarrow | \text{)}$$

If a transcript were longer than $T - 2$ frames it is truncated (essentially never fires at 50 fps). The label vector is exactly T long: every frame has a class.

The duration budget - the 'frames-per-second' idea. Grading all T frames would make ~70% of the training signal 'predict `<fill>`', and the model can drift toward the majority class. Instead only the first n_{keep} frames are graded:

$$n_{\text{keep}} = \lceil \text{dur}_s \times 25 \rceil \quad y_t \leftarrow -100 \text{ for } t \geq n_{\text{keep}} \text{ or } t \text{ is label padding}$$

25 graded frames per second is a deliberate midpoint: comfortably above any real character rate (~12-18 chars/s, so the transcript always fits), yet only half the 50 fps frame rate - so the model still learns a stretch of graded `<fill>` after `</s>` (it must know when speech *content ends*), while the far tail costs nothing. Frames beyond n_{keep} are trained on **nothing at all** - which becomes important at decode time (Section 9).

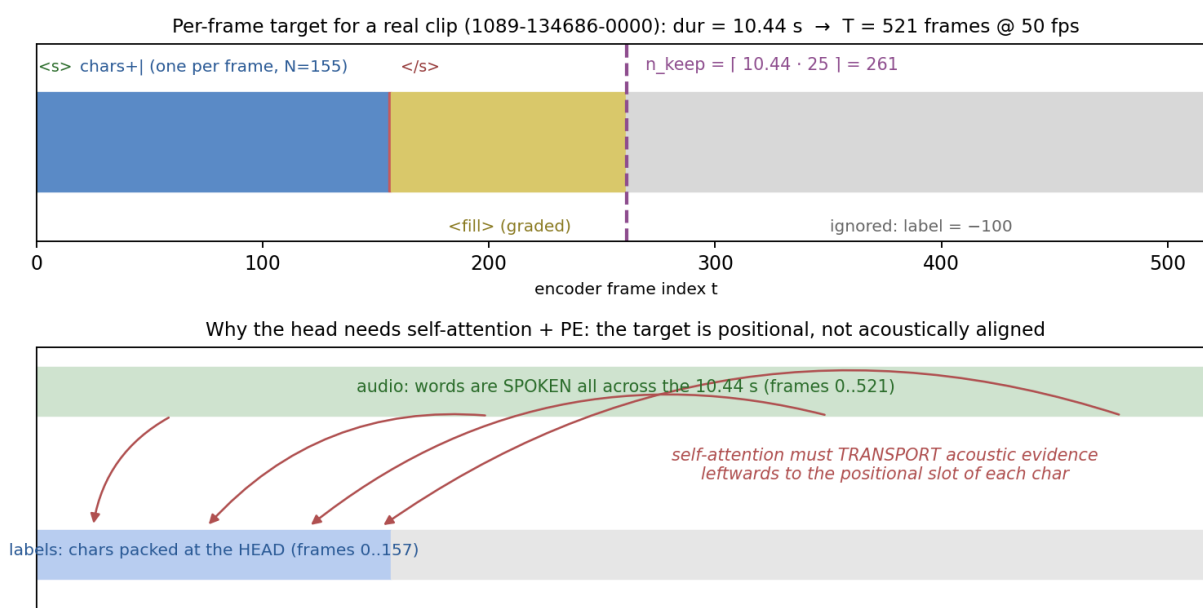


Figure 3 - top: the actual label layout for the 10.44 s worked example ($T = 521$, $n_{\text{keep}} = 261$, 157 head labels). Bottom: why this is hard - sounds are spread over the whole clip but labels sit at the head; attention must carry evidence leftwards.

The three masks - do not confuse them. The reference is emphatic about this and it is the most common source of confusion in the codebase:

mask	lives where	mechanism	purpose
1. frame / attention mask	inside every MHSA	key-padding scores set to -inf	batch-padding frames of short clips are invisible to attention

mask	lives where	mechanism	purpose
2. loss mask	labels tensor	label = -100, CE ignore_index	(a) label padding and (b) all frames past n_keep are simply not graded
3. input masking (OFF here)	on the HuBERT tap, before PE	200 ms spans replaced by a learned mask_embed	optional SpecAugment-style regulariser: the SA stack must reconstruct masked content from context

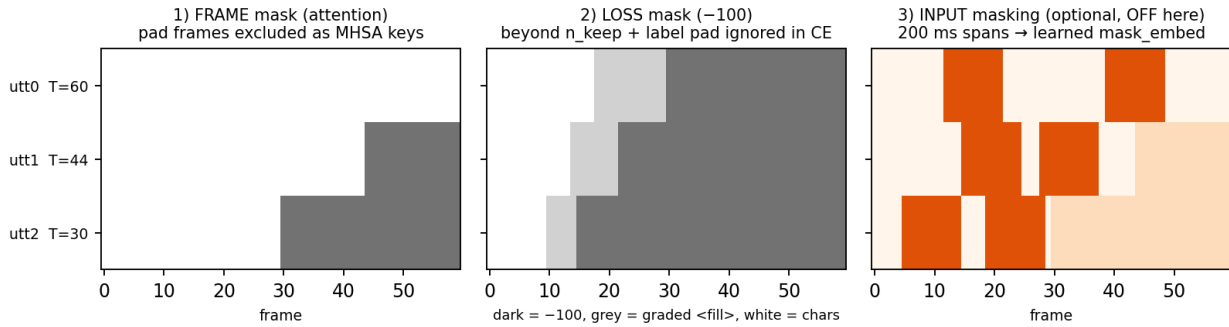


Figure 4 - the three masks on a toy 3-clip batch. Only masks 1 and 2 are active in this run ($\text{mask_prob} = 0$).

7 Block 5 - the loss: framewise cross-entropy

Given logits z (B, T, 33) and labels y (B, T), both flattened:

$$\mathcal{L} = -\frac{1}{|G|} \sum_{(b,t) \in G} \log \text{softmax}(z_{b,t})_{y_{b,t}} \quad G = \{(b,t) : y_{b,t} \neq -100\} \quad z_{b,t} \in \mathbb{R}^{33}$$

That is the entire loss - one softmax per frame, averaged over graded frames G . Sanity anchor used by the smoke test: an untrained head should score about the entropy of a uniform guess,

$$\mathcal{L}_{\text{init}} \approx -\log \frac{1}{33} = \ln 33 \approx 3.497$$

and the observed initial loss ~4.05 in this run's step-0 eval is in that ballpark (slightly higher because the re-initialised head is not exactly uniform). **Class balance:** inside the graded region roughly 60% of frames are content and 40% are `<fill>` (for the worked example: 157 of 261 vs 104 of 261) - mild enough that the implemented training uses **unweighted** CE. A `fill_weight` knob exists to down-weight `<fill>` if a configuration collapses to all-filler; it stayed unused here, and the trained model's eval `fill_pred_rate` of ~0.41 confirms it predicts filler at almost exactly the target rate.

8 Blocks 6-8 - wiring, smoke test, and the overfit proof

Block 6 - build_model(). Loads the pretrained config, overwrites vocab_size to 33 and attaches all SA/PE/masking hyper-parameters to it, then calls from_pretrained(..., ignore_mismatched_sizes=True). The pretrained checkpoint is a CTC model with a 32-class head; the flag lets the 33-class lm_head be freshly re-initialised instead of erroring. SpecAugment is disabled for determinism. Finally the freeze helpers set the trainable set: SA + lm_head only.

Block 7 - smoke test. One collated batch, one forward, one loss. It prints shapes, trainable-parameter count and the loss vs the ln 33 reference - a 30-second check that tokenizer, collator, masks and model agree about T before any GPU-hours are spent.

Block 8 - the overfit driver. The classic learnability experiment: take N=10 clips, full-batch AdamW (lr 2e-3), a few hundred epochs. If the architecture + target are coherent, loss must go to ~0 and the model must reproduce the graded transcripts verbatim; if the target were unlearnable, it plateaus. Running it with and without --pe is how the PE requirement was established empirically: same data, same head, PE off fails to memorise even 10 clips, PE on reproduces them. _decode() maps argmax frame ids to text by dropping specials/<fill> and turning | into spaces - the same stripping used everywhere downstream.

9 The training run: full960_sa8_pe_fps25_lin_100ep_bs32acc2

The run trains the reference architecture (imported **unchanged** from filler_sa_reference.py by train.py - model, collator, loss are the same objects) on the full 960 h manifest with no feature caching: the frozen encoder runs a fresh forward every step.

knob	value	notes
encoder	hubert-xlarge-ls960-ft	frozen; ~964M params, already ASR-finetuned (its own CTC head discarded)
head	8 SA layers + lm_head, PE on	157.46M trainable
objective	framewise CE, fps budget 25	unweighted; ignore_index -100
data	281,241 train utts / 500 dev-clean utts	lazy audio loading, 8 dataloader workers per rank
parallelism	6-GPU DDP (torchrun), bf16 autocast	physical GPUs 1,4,6,7,8,9; NCCL over loopback
batch	32 per GPU x 2 accum x 6 GPUs = 384	grad accumulation with no_sync() between boundaries
optimizer	AdamW lr 2e-4, betas (0.9, 0.98), eps 1e-6, wd 0.005	grad-norm clip 1.0
schedule	linear: 2,000-step warmup then decay to 0	sized to 73,200 total steps ('lin' in the run name)
epochs / steps	100 epochs = 73,200 steps	732 steps/epoch (see arithmetic below)
eval / ckpt	every 2,000 steps on 500 dev clips	best.pt on dev-WER improvement; latest.pt for resume

$$B_{\text{eff}} = 32_{\text{batch}} \times 2_{\text{accum}} \times 6_{\text{GPUs}} = 384 \quad \text{steps/epoch} = \left\lfloor \frac{281241/6}{32} \right\rfloor / 2 = 732 \quad 100 \times 732 = 73,200 \text{ steps}$$

$$\eta(s) = \eta_{\text{max}} \cdot \frac{s}{2000} \quad (s \leq 2000) \quad \eta(s) = \eta_{\text{max}} \cdot \frac{73200 - s}{73200 - 2000} \quad (s > 2000) \quad \eta_{\text{max}} = 2 \times 10^{-4}$$

DDP mechanics worth noting. Each rank owns 1/6 of the data via DistributedSampler (reshuffled every epoch). During the first of each pair of micro-batches the loop wraps backward in model.no_sync() so gradients are only all-reduced at the accumulation boundary - halving communication. The loss is divided by the accumulation factor so gradient magnitudes match a true batch-384 step. Forward runs in bf16 autocast; logits are cast back to fp32 before the CE for numerical safety. Checkpoints store **only head keys** (sa.*, lm_head.*) plus optimizer/scheduler state - the frozen HuBERT never changes, so it is

reloaded from the pretrained checkpoint at inference and the head is applied with `strict=False`.

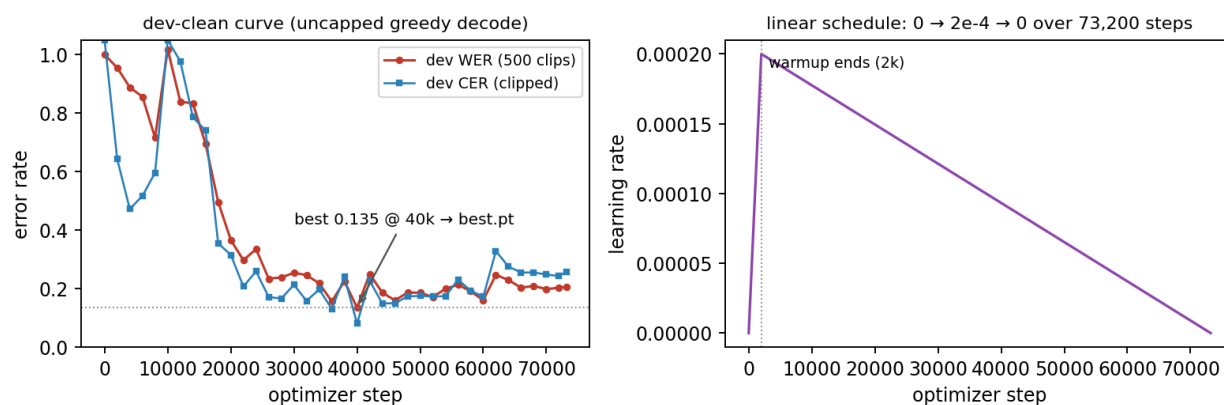


Figure 5 - left: dev-clean WER/CER over the run (evaluated every 2,000 steps with UNCAPPED greedy decode, hence pessimistic and noisy - see Section 9). Best dev WER 0.135 at step 40k became best.pt. Right: the linear LR schedule.

How the run actually unfolded. At step 0 the untrained head scores WER 1.0 and CER 7.96 - it emits near-random characters over all 521 frames, so hypotheses are many times longer than references (CER can exceed 1). Through warmup the filler structure is learned first (fill_pred_rate climbs to ~0.41 and stays pinned there), then content accuracy grinds upward for tens of thousands of steps - this is the slow part: the head is learning to route evidence across time. Dev WER bottoms out at **0.135 around step 40k (~epoch 55)**. After that the curve oscillates and drifts up while prediction entropy keeps falling (0.15 → 0.07): the classic signature of a head over-sharpening on a converged schedule - more epochs stopped helping. Because best.pt snapshots only on dev-WER improvement, the over-run wasted compute but not quality.

10 Inference: greedy decode, and why it must be capped

Decoding is deliberately primitive - **argmax per frame**, no beam, no language model:

```
logits = model(input_values, attention_mask).logits          # (B, T, 33)
pred   = logits.argmax(-1)                                # (B, T) int ids
hit    = (pred == eos_id).nonzero()                        # first </s> = model's own end estimate
capped = pred[: hit[0]]                                    # ... or pred[: n_keep] - dump_decodes.py uses min(first </s>, n_keep)
hyp    = tok.decode(capped, skip_special_tokens=True, group_tokens=False) # NO CTC collapse
hyp    = hyp.replace('|', ' ')
```

Why cap at all? Every frame past `n_keep` was labelled -100 during training - the model received *zero* gradient about them. Whatever it emits there is undefined behaviour, and in practice it leaks stray characters. An uncapped decode therefore appends garbage after the real transcript and inflates WER badly (this is exactly why the in-training eval curve in Figure 5 is noisy and pessimistic - `evaluate()` decodes uncapped). The fix costs one line: cut at the first `</s>` the model emits (its own duration estimate) or at `n_keep`, whichever comes first.

Why `group_tokens=False` (no CTC-style duplicate merging)? In CTC, consecutive repeats are alignment artefacts and must be collapsed. Here the target places *exactly one* character per frame - a repeated letter in the output is a genuine double letter ('ll' in 'hello'), and collapsing destroys it. The decode-variant sweep on this run's best.pt proves the point empirically:

decode variant	WER	CER	SER	verdict
cut at first <code></s></code> (no collapse)	12.54%	5.66%	36.0%	best - used everywhere
cut at <code></s></code> + collapse duplicates	20.59%	7.41%	81.9%	collapse merges real double letters
cut at first run of 3+ <code><fill></code>	12.54%	5.66%	36.0%	equivalent end detector
<code></s></code> or fill-run, whichever first	12.54%	5.66%	36.0%	ditto

A real decode from this run (test-clean, greedy, capped). The model emits the frame stream on top; stripping specials and mapping `|` to spaces yields the hypothesis:

```
frames : <s> h e | h o p e d | t h e r e | w o u l d | b e | s t e w ... s a u c e </s> <fill> x260
HYP    : he hoped there would be stew for dinner turnips and carrots and bruised
        potatoes and fat mutton pieces to be ladled out in thick peppered flour
        fattened sauce                                     (WER 0.000)
-----
HYP    : hello berty any good in your mind                REF: hello bertie ... (1 word err)
```

The second clip shows the characteristic error mode: *berty* for *bertie* - acoustically perfect, orthographically wrong. A char-level greedy decoder has no lexicon or language model to prefer the correct spelling, which is why WER (12.5%) is more than double CER (5.7%): errors are mostly single-letter misspellings inside otherwise-correct words. This is also why a downstream LLM-correction pass over these decodes is a natural add-on (explored elsewhere in this project).

11 Results, honest limitations, and where to go next

metric (test-clean, 2,620 utts, greedy + <code></s></code> cap)	value
Word Error Rate (WER)	12.54%
Character Error Rate (CER)	5.66%
Sentence Error Rate (SER)	35.99%
best dev-clean WER during training (uncapped eval)	13.5% @ step 40,000
trainable / total parameters	157.46M / ~1.12B (~14%)

For calibration: the same frozen encoder *with its native CTC head* sits around ~2% WER on test-clean. The gap is the price of the deliberately non-acoustic target - and simultaneously the interesting finding: a frozen encoder plus 8 attention layers can solve 'transcribe by writing characters into indexed slots' to within 12.5% WER using nothing but per-frame CE.

Known limitations (from the code comments and the eval analysis):

- **Positional pinning is non-acoustic.** Character *i* sits at frame *i* regardless of when it is spoken, so the head must effectively count characters while listening. Errors compound rightward: one wrong slot early shifts everything after it (a big contributor to SER).
- **Greedy + no LM.** Spelling errors (berty/bertie) go uncorrected; a lexicon-constrained or LLM-rescored decode would recover many of them cheaply.
- **In-training eval is uncapped**, so the Figure-5 curve overstates error and adds noise; model selection still worked, but capped eval would give a cleaner signal (and was later implemented in the eval_analysis / dump_decodes tooling).
- **Long clips stress the design:** transport distance grows with *T* while the graded-tail fraction shrinks - the truncation guard exists but the geometry gets harder.

Natural next steps, all wired but unused in this run: train-time input masking (`mask_prob > 0`) as a regulariser; gradual unfreezing of the top HuBERT layers with the discriminative LR path; a `fill_weight` sweep; and LLM-based generative error correction over the capped decodes.

Appendix A - file map

file	role
filler_asr/filler_sa_reference.py	single-file reference: Blocks 0-8 (model, tokenizer, collator, loss, smoke, overfit)
train_sa_pe_lm_head/train.py	full-data training loop: DDP, bf16, grad accum, eval, best/latest checkpoints, wandb
train_sa_pe_lm_head/data.py	manifest loader + re-export of the reference collator
train_sa_pe_lm_head/run.sh	launcher: env, GPU pinning (PCI_BUS_ID), NCCL loopback, torchrun
train_sa_pe_lm_head/train_sa8_100ep_6gpu.sh	THIS run's launcher: tmux-detached, GPU/ckpt-clobber preflights, 6 GPUs
train_sa_pe_lm_head/infer.py	decode engine: rebuilds arch from ckpt keys, </s>-capped greedy, WER/CER/SER
train_sa_pe_lm_head/dump_decodes.py	per-utterance dump: RAW vs NKEEP frame streams + capped hyp vs ref + variants
runs/full960_sa8_pe_fps25_lin_100ep_bs32acc2/	best.pt / latest.pt (1.9 GB: head + AdamW state), tokenizer files, decode dumps + summaries

Appendix B - reproducing the run

```
# smoke test (30 s sanity check of shapes + ln(33) loss)
cd /speech/tomson/filler_asr
python filler_sa_reference.py --n 4 --pe

# overfit proof (should reproduce all 10 clips)
python filler_sa_reference.py --train --n 10 --epochs 200 --pe

# the full 100-epoch, 6-GPU run (tmux-detached; preflights GPUs + out dir)
cd /speech/tomson/filler_asr/train_sa_pe_lm_head
bash train_sa8_100ep_6gpu.sh          # GPUS=1,4,6,7,8,9  EPOCHS=100  BATCH=32  ACCUM=2

# decode test-clean with the trained head
python infer.py --run_dir runs/full960_sa8_pe_fps25_lin_100ep_bs32acc2 \
    --manifest ../librispeech_test_clean.jsonl
```

```
# loading the trained head for your own inference
from filler_sa_reference import build_tokenizer, build_model
import torch
run = 'runs/full960_sa8_pe_fps25_lin_100ep_bs32acc2'
tok = build_tokenizer(run)                # tokenizer was saved into the run dir
ck = torch.load(f'{run}/best.pt', map_location='cpu')
model = build_model('/speech/tomson/filler_asr/models/hubert-xlarge-ls960-ft',
                    tok, num_sa_layers=8, use_sinusoidal_pe=True, device='cuda')
model.load_state_dict(ck['head'], strict=False) # head keys only; hubert already loaded
model.eval()
```

Generated from the source tree on 2026-07-05. All numbers are read from the run's own logs and decode summaries (train_sa8_100ep_.log, librispeech_test_clean.dump.summary / .strip_variants.summary).*